

Harness 七层架构：模型外面那七层，才是 AI 好不好用的关键

强西 强西 2026年4月18日 11:04 浙江

之前聊了一个概念，编排工程（Harness Engineering）。

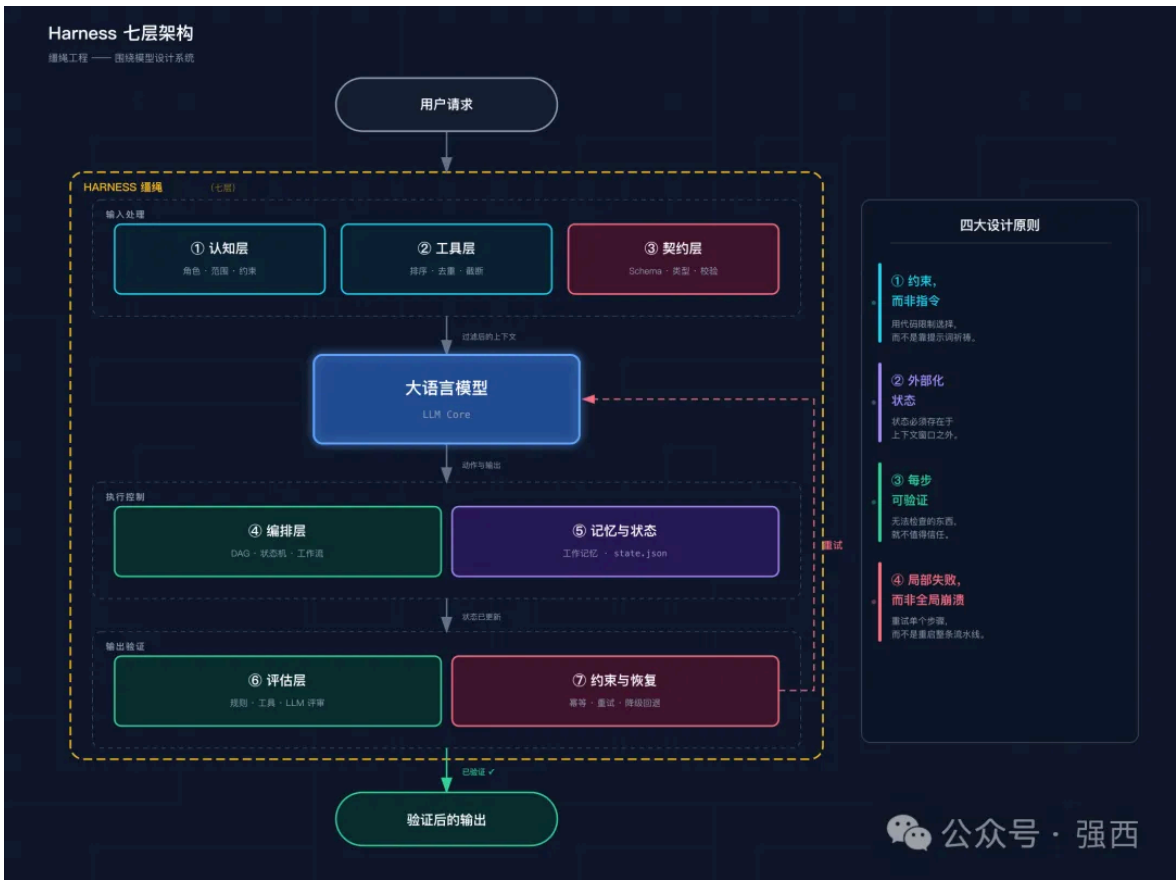
核心观点就一句话：同一个模型，在不同系统里表现天差地别，差距不在模型，在模型外面那一圈东西。

那篇发出去之后，后台收到最多的一类留言是：道理我懂了，但具体是哪些东西？我该从哪下手？

今天就把这件事掰开了讲。

我整理了一张架构图，把编排工程拆成了七层。每一层是什么、干什么、在 Claude Code 里长什么样，全部对应到实操。看完这张图，你就知道自己目前卡在哪一层，该往哪使劲。

先看全貌：Harness 七层架构



整个流程是这样的：用户请求进来，先经过三层输入处理（认知层、工具层、契约层），过滤和结构化之后，喂给中间的大语言模型。模型产出动作和输出，交给执行控制层

（编排层、记忆与状态层）去调度。最后经过输出验证（评估层、约束与恢复层），确认没问题了，才输出给用户。

七层，三个阶段，一条完整的流水线。

不是每一层都需要你从零搭建。如果你用的是 Claude Code 这类成熟工具，很多层它已经帮你做了。但你得知道它们的存在，才能在对的地方使劲。

下面一层一层拆。

输入处理：模型看到什么，决定它能做什么

第一层：认知层 — 角色·范围·约束

这层解决的是一个最基本的问题：模型知不知道自己是谁、该干什么、不该干什么。

听起来简单，但大部分人用 AI 的时候，这层是空的。打开对话框就开始提需求，模型完全不知道你的项目背景、技术栈、代码规范。它只能靠猜。

在 Claude Code 里，认知层对应的就是 CLAUDE.md。

你在项目根目录放一个 CLAUDE.md，写清楚：这个项目用什么技术栈、代码风格是什么、有哪些不能碰的东西、测试怎么跑。模型每次启动都会读到这些信息，相当于你给它发了一份入职手册。

我自己的 CLAUDE.md 里会写这些：项目是 Tauri v2 + React 19 + Rust，前端用 TypeScript 严格模式，组件用函数式写法，不用 class。这些写了之后，模型生成的代码风格就跟项目一致了，不用每次手动纠正。

认知层做得好不好，直接决定了后面所有层的起点。就像你给新员工的入职培训，第一天没讲清楚，后面天天在纠错。

第二层：工具层 — 排序·去重·截断

模型不是只靠「想」来干活的，它还需要「动手」。工具层管的就是：模型能用哪些工具，工具返回的信息怎么处理。

这里有个关键词：截断。

模型的上下文窗口是有限的。你让它搜一个代码库，搜出来 500 个结果，全塞进去？模型直接懵了。工具层要做的就是排序、去重、截断，把最相关的信息筛出来，控制在模型能消化的范围内。

Claude Code 内置了一整套工具：读文件、写文件、跑终端命令、搜代码、Glob 匹配、LSP 类型查询。你不需要自己搭，但你需要知道它们的存在，才能在对话里有效地引导模型去用。

比如你说「帮我找一下所有用到 getUserInfo 的地方」，模型会自动调用 Grep 工具去搜。但如果你说「帮我优化一下代码」，它可能不知道该从哪开始找。你给的指令越具体，工具层的效率越高。

第三层：契约层 — Schema·类型·校验

这层是很多人忽略的，但它可能是最影响输出质量的一层。

契约层做的事情是：规定模型的输出必须符合什么格式。不是靠 prompt 里写「请用 JSON 格式返回」这种祈祷式约束，而是用 Schema 做硬性校验。

打个比方。你让模型帮你生成一个 API 接口的返回值，你在 prompt 里说「请返回 JSON」，它可能返回一个 JSON，也可能返回一段解释文字加一个 JSON，还可能返回一个格式不对的 JSON。这就是「指令」的局限性，模型可以选择不听。

但如果你用 Schema 约束，定义好返回值必须是{ name: string, age: number }这个结构，模型的输出会被强制校验。不符合？打回重来。

这就是图里写的「约束，而非指令」。用代码限制选择，而不是靠提示词祈祷。

在 Claude Code 的场景里，契约层体现在 hooks。你可以设置 pre-commit hook，模型每次要提交代码之前，自动跑一遍 lint 和类型检查。不通过？不让提交。这就是用硬约束替代软指令。

三层输入处理讲完了。认知层告诉模型「你是谁」，工具层告诉模型「你能用什么」，契约层告诉模型「你的输出必须长什么样」。三层过滤完，喂给模型的就不再是一堆原始噪音，而是结构化的、有边界的上下文。

LLM Core：模型本身

中间这层就是大语言模型本身。

注意它在架构图里的位置：被上面三层和下面四层包裹着。模型是核心，但它不是全部。它接收过滤后的上下文，产出动作和输出，然后交给下面的层去执行和验证。

这也是编排工程最核心的洞察：模型的能力上限，很大程度上取决于包裹它的那些层做得好不好。同一个 Opus 模型，裸跑和套上完整 Harness 之后，表现完全是两个量级。

执行控制：模型的手和脚

第四层：编排层 — DAG·状态机·工作流

这层是整个架构的调度中心。

模型产出了一个动作，比如「我要改这个文件」，但改完之后呢？要不要跑测试？测试挂了怎么办？要不要通知用户？这些「下一步做什么」的决策，就是编排层在管。

简单任务可以是线性的：读代码 → 改代码 → 跑测试 → 完成。但复杂任务往往是一个 DAG（有向无环图），有分支、有并行、有条件判断。

Claude Code 里你能直观感受到这层。你让它「给这个项目加一个暗黑模式」，它不会一步到位。它会先搜现有的样式系统，再看有没有主题变量，然后决定是改 CSS 变量还是加一套新的主题配置，改完之后跑一下看看有没有样式冲突。每一步的结果决定下一步怎么走。

这个过程你看起来像是模型在「思考」，其实是编排层在根据每一步的输出，动态决定下一步的路径。

第五层：记忆与状态 — 工作记忆·state.json

这层解决一个很现实的问题：模型会忘事。

上下文窗口再大也有限。一个复杂任务聊了几十轮之后，早期的信息会被压缩甚至丢弃。如果没有外部状态管理，模型做到第十步可能已经忘了第一步的结论。

图里标注的是「外部化状态」，意思是：状态必须存在于上下文窗口之外。

Claude Code 的 memory 系统就是这层的实现。它会把你的偏好、项目信息、你给过的反馈写成 Markdown 文件存在本地。下次开新对话，这些信息自动加载。模型不需要从头了解你，它已经「记得」你是谁。

还有一个更细的层面：任务内的工作记忆。Claude Code 在执行复杂任务时，会用 TodoList 追踪进度，哪些步骤做完了，哪些还没做，当前卡在哪。这就是 state.json 的概念，把任务状态外部化，不依赖模型自己的「脑子」。

这层做好了，AI 就从一个「金鱼记忆」的聊天机器人，变成了一个能持续跟进项目的协作者。

输出验证：不是做完就算，做对才算

第六层：评估层 — 规则·工具·LLM 评审

模型干完活了，但干得对不对？这层就是质检员。

评估可以分三个级别。最轻量的是规则检查：代码能不能通过 lint？类型对不对？有没有语法错误？这些是确定性的，跑一遍就知道。

中间一级是工具检查：跑一遍单元测试，看看有没有挂的。build 一下，看看能不能编译通过。

最重的一级是 LLM 评审：用另一个模型（或者同一个模型换个角色）来审查输出质量。这就是上篇文章提到的「生产与验收分离」，不能既当运动员又当裁判。

Claude Code 在这层做得很自然。它改完代码会自己跑测试，测试挂了会自己看报错、分析原因、再改一版。你不需要手动触发这个循环，它内置了「改 → 验 → 修」的闭环。

但如果你想加强这层，可以在 CLAUDE.md 里写明：每次改完代码必须跑 npm test，必须通过 TypeScript 类型检查。这就是把评估标准显式化，不靠模型自觉，靠规则兜底。

第七层：约束与恢复 — 幂等·重试·降级回退

这是最后一层，也是最容易被忽略的一层。

前面六层都在讲「怎么把事情做好」，这层讲的是「事情搞砸了怎么办」。

真实环境里，失败是常态。API 超时、文件冲突、模型幻觉、网络断了。。。一个只能在理想条件下运行的系统，不是一个能用的系统。

这层的核心思想是图里写的第四个设计原则：「局部失败，而非全局崩溃」。重试单个步骤，而不是重启整条流水线。

举个例子。模型在改一个文件的时候，发现文件被别人改过了，产生了冲突。一个没有恢复机制的系统会直接报错停下来，等你来处理。但一个有恢复层的系统会：先回退这一步的修改，重新读取最新文件，基于新内容再改一次。

Claude Code 里你能观察到这个行为。它改代码改出了编译错误，不会直接放弃，会自己看报错、定位问题、修复、再试。如果连续几次都修不好，它会降级，告诉你「这个问题我搞不定，可能需要你看一下」，而不是在那死循环。

幂等、重试、降级回退，这三个词听起来像后端工程师的黑话，但它们在 AI 系统里同样重要。一个能自己从错误中恢复的系统，才是一个你敢放手让它干活的系统。

四大设计原则：贯穿七层的底层逻辑

七层架构是「怎么搭」，四大设计原则是「怎么想」。图的右侧列了四条，每一条都值得单独说说。

原则一：约束，而非指令。

用代码限制选择，而不是靠提示词祈祷。

这条贯穿了整个架构。契约层的 Schema 校验是约束，评估层的自动测试是约束，hooks 的强制检查是约束。你越是把规则写成代码，模型就越难跑偏。

反过来，你越是依赖 prompt 里的「请注意」「务必」「一定要」，模型就越容易选择性失聪。

原则二：外部化状态。

状态必须存在于上下文窗口之外。

模型的上下文窗口是它的「工作台」，东西放多了就乱，放久了就丢。重要的状态，任务进度、用户偏好、中间结果，必须写到外面去。文件系统、数据库、state.json，都行。

Claude Code 的 memory 文件、TodoList、甚至 git 本身，都是外部化状态的载体。模型可以随时读取，不怕丢。

原则三：每步可验证。

无法检查的东西，就不值得信任。

这条是评估层的灵魂。模型每做一步，都应该有一个可以验证的标准。代码改了？跑测试。文件写了？检查格式。API 调了？看返回值。

如果一个步骤没有验证手段，那你就不应该让模型自动执行它。要么加一个验证机制，要么改成需要人工确认的步骤。

原则四：局部失败，而非全局崩溃。

重试单个步骤，而不是重启整条流水线。

这条是约束与恢复层的灵魂。一个十步的任务，第七步挂了，你不需要从头来。回退第七步，修复问题，从第七步继续。

这要求每一步都是相对独立的，有明确的输入输出，失败了可以单独重试。这也是为什么编排层要把任务拆成 DAG 而不是一坨意大利面代码。

回到你身上：从哪一层开始？

七层听起来很多，但你不需要一次全搞定。

如果你现在用 AI 还是「打开对话框直接聊」的状态，先把第一层做了。写一个 CLAUDE.md，把项目背景和基本规范写清楚。光这一步，体验就会有明显提升。

如果你已经在用 Claude Code 但觉得「不够稳」，看看第三层和第六层。加 hooks 做硬约束，在 CLAUDE.md 里写明验证标准。

如果你在搭自己的 AI 工作流或者 Agent 系统，那七层都得考虑，尤其是第五层（记忆与状态）和第七层（约束与恢复），这两层是「能用」和「好用」之间的分水岭。

记住一句话：模型是发动机，Harness 是整辆车。你不需要换一台更大的发动机，你需要的是把车造好。

以上，既然看到这里了，如果觉得不错，随手点个赞、在看、转发三连吧，如果想第一时间收到推送，也可以给我个星标★～

谢谢你看我的文章，我们，下次再见。